04_knowledge_based

August 14, 2026

1 Programming Assignment 1: Recommender System

1.0.1 Coure: CS373

Developer 1: Jalin A. Brown

Developer 2: Sandeep Durga

File: 04_knowledge_based.ipynb

1.0.2 Part Three: Knowledge-Based Recommender

1. Create preference elicitation interface:

- Collect user preferences for: Genres: Multi-select from MovieLens categories (Action, Comedy, Drama, etc.) Release years: Min and max year range Rating threshold: Minimum acceptable average rating (1-5) Runtime: Short (<90min), Medium (90-120min), Long (>120min), Any

2. Parse movie metadata:

- Extract genres from movie data (pipe-separated format)
- Parse release years from movie titles
- Calculate average ratings for each movie from training data
- Handle missing runtime data

3. Implement content-based scoring:

- Genre matching (50% weight): Calculate overlap using set intersection: score = len(movie_genres preferred_genres) / len(preferred_genres) * 40
- Year preference (30% weight)
- Rating filter (20% weight)
- Total score = sum of all components (max 100)

4. Generate recommendations with explanations:

- Create 3 different user preference profiles
- Generate top 3 recommendations with explanations for each profile
- Analyze recommendation quality
- Return top 3 movies with explanations of why each was selected Example: "Recommended because: matches Action+Sci-Fi genres, 1999 release in your range, 4.3/5 rating above threshold"

2 1: Create preference elicitation interface

[69]:
```
# Import movie and rating data to prepare for metadata parsing
import numpy as np
import pandas as pd

# Movie metadata (MovieLens 100K)
# # tab separated list of
#               movie id | movie title | release date | video release date |
#               IMDb URL | unknown | Action | Adventure | Animation |
#               Children's | Comedy | Crime | Documentary | Drama | Fantasy|
#               Film-Noir | Horror | Musical | Mystery | Romance | Sci-Fi |
#               Thriller | War | Western |
#               The last 19 fields are the genres, a 1 indicates the movie
#               is of that genre, a 0 indicates it is not; movies can be in
#               several genres at once.
#               The movie ids are the ones used in the u.data data set.
```

[70]:
```
# Load metadata from u.item
u_data = pd.read_csv("../data/u.data", sep="\t",␣
 ↪names=["user_id","item_id","rating","timestamp"])

u_item_cols = [
    "movie_id", "movie_title", "release_date", "video_release_date", "IMDb_URL",
    "unknown","Action","Adventure","Animation","Children","Comedy","Crime",
    "Documentary","Drama","Fantasy","Film-Noir","Horror","Musical","Mystery",
    "Romance","Sci-Fi","Thriller","War","Western"
]

u_item = pd.read_csv("../data/u.item", sep="|", header=None, names=u_item_cols,␣
 ↪encoding="latin-1")
# Drop duplicate movie titles, keeping the first occurrence
u_item = u_item.drop_duplicates(subset="movie_title", keep="first").
 ↪reset_index(drop=True)

genres_cols = [
    "Action","Adventure","Animation","Children","Comedy","Crime","Documentary",
    "Drama","Fantasy","Film-Noir","Horror","Musical","Mystery","Romance",
    "Sci-Fi","Thriller","War","Western"
]


print("Movies loaded:", len(u_item))
print("Ratings loaded:", len(u_data))
u_item.head(20)
```

```
Movies loaded: 1664
Ratings loaded: 100000
```

[70]:

	movie_id	movie_title	release_date
0	1	Toy Story (1995)	01-Jan-1995
1	2	GoldenEye (1995)	01-Jan-1995
2	3	Four Rooms (1995)	01-Jan-1995
3	4	Get Shorty (1995)	01-Jan-1995
4	5	Copycat (1995)	01-Jan-1995
5	6	Shanghai Triad (Yao a yao yao dao waipo qiao) …	01-Jan-1995
6	7	Twelve Monkeys (1995)	01-Jan-1995
7	8	Babe (1995)	01-Jan-1995
8	9	Dead Man Walking (1995)	01-Jan-1995
9	10	Richard III (1995)	22-Jan-1996
10	11	Seven (Se7en) (1995)	01-Jan-1995
11	12	Usual Suspects, The (1995)	14-Aug-1995
12	13	Mighty Aphrodite (1995)	30-Oct-1995
13	14	Postino, Il (1994)	01-Jan-1994
14	15	Mr. Holland's Opus (1995)	29-Jan-1996
15	16	French Twist (Gazon maudit) (1995)	01-Jan-1995
16	17	From Dusk Till Dawn (1996)	05-Feb-1996
17	18	White Balloon, The (1995)	01-Jan-1995
18	19	Antonia's Line (1995)	01-Jan-1995
19	20	Angels and Insects (1995)	01-Jan-1995

	video_release_date	IMDb_URL
0	NaN	http://us.imdb.com/M/title-exact?Toy%20Story%2…
1	NaN	http://us.imdb.com/M/title-exact?GoldenEye%20(…
2	NaN	http://us.imdb.com/M/title-exact?Four%20Rooms%…
3	NaN	http://us.imdb.com/M/title-exact?Get%20Shorty%…
4	NaN	http://us.imdb.com/M/title-exact?Copycat%20(1995)
5	NaN	http://us.imdb.com/Title?Yao+a+yao+yao+dao+wai…
6	NaN	http://us.imdb.com/M/title-exact?Twelve%20Monk…
7	NaN	http://us.imdb.com/M/title-exact?Babe%20(1995)
8	NaN	http://us.imdb.com/M/title-exact?Dead%20Man%20…
9	NaN	http://us.imdb.com/M/title-exact?Richard%20III…
10	NaN	http://us.imdb.com/M/title-exact?Se7en%20(1995)
11	NaN	http://us.imdb.com/M/title-exact?Usual%20Suspe…
12	NaN	http://us.imdb.com/M/title-exact?Mighty%20Aphr…
13	NaN	http://us.imdb.com/M/title-exact?Postino,%20Il…
14	NaN	http://us.imdb.com/M/title-exact?Mr.%20Holland…
15	NaN	http://us.imdb.com/M/title-exact?Gazon%20maudi…
16	NaN	http://us.imdb.com/M/title-exact?From%20Dusk%2…
17	NaN	http://us.imdb.com/M/title-exact?Badkonake%20S…
18	NaN	http://us.imdb.com/M/title-exact?Antonia%20(1995)
19	NaN	http://us.imdb.com/M/title-exact?Angels%20and%…

	unknown	Action	Adventure	Animation	Children	…	Fantasy	Film-Noir
0	0	0	0	1	1	…	0	0
1	0	1	1	0	0	…	0	0

2	0	0	0	0	0	…	0	0
3	0	1	0	0	0	…	0	0
4	0	0	0	0	0	…	0	0
5	0	0	0	0	0	…	0	0
6	0	0	0	0	0	…	0	0
7	0	0	0	0	1	…	0	0
8	0	0	0	0	0	…	0	0
9	0	0	0	0	0	…	0	0
10	0	0	0	0	0	…	0	0
11	0	0	0	0	0	…	0	0
12	0	0	0	0	0	…	0	0
13	0	0	0	0	0	…	0	0
14	0	0	0	0	0	…	0	0
15	0	0	0	0	0	…	0	0
16	0	1	0	0	0	…	0	0
17	0	0	0	0	0	…	0	0
18	0	0	0	0	0	…	0	0
19	0	0	0	0	0	…	0	0

	Horror	Musical	Mystery	Romance	Sci-Fi	Thriller	War	Western
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	0	0
2	0	0	0	0	0	1	0	0
3	0	0	0	0	0	0	0	0
4	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0
6	0	0	0	0	1	0	0	0
7	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	1	0
10	0	0	0	0	0	1	0	0
11	0	0	0	0	0	1	0	0
12	0	0	0	0	0	0	0	0
13	0	0	0	1	0	0	0	0
14	0	0	0	0	0	0	0	0
15	0	0	0	1	0	0	0	0
16	1	0	0	0	0	1	0	0
17	0	0	0	0	0	0	0	0
18	0	0	0	0	0	0	0	0
19	0	0	0	1	0	0	0	0

[20 rows x 24 columns]

2.0.1 2: Parse movie metadata

[71]:
```
# Parse the release year from movie titles
u_item["release_year"] = u_item["movie_title"].str.extract(r"\((\d{4})\)").
 ↪astype(float)

# Genre list column
genre_list = []
for i, row in u_item.iterrows():
    genres = []
    for g in genres_cols:
        if int(row[g]) == 1:
            genres.append(g)
    genre_list.append(genres)
u_item["genre_list"] = genre_list

# Create a synthetic runtime column
np.random.seed(0)
u_item["runtime_min"] = np.random.choice(
    [60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190, 200, np.
 ↪nan],
    size=len(u_item),
    p=[0.01, 0.03, 0.08, 0.15, 0.18, 0.18, 0.15, 0.08, 0.04, 0.03, 0.02, 0.015,␣
 ↪0.01, 0.005, 0.004, 0.016]
)

runtime_cat = []
for rt in u_item["runtime_min"]:
    if pd.isna(rt):
        runtime_cat.append("unknown")
    elif rt < 90:
        runtime_cat.append("short")
    elif rt <= 120:
        runtime_cat.append("medium")
    else:
        runtime_cat.append("long")
u_item["runtime_category"] = runtime_cat
```

[72]:
```
# Calculate average ratings for each movie from training data
rating_average = ( u_data.groupby("item_id")["rating"].mean().
 ↪rename("rating_average").astype(float).reset_index())
```

[73]:
```
# Combine all into one matrix
movies_matrix = pd.merge(
    u_item[["movie_id", "movie_title", "release_year", "genre_list",␣
 ↪"runtime_category"]],
    rating_average,
```

```
    left_on="movie_id",
    right_on="item_id",
    how="left"
).drop(columns="item_id")

# Handle missing average ratings with global mean
global_mean = u_data["rating"].mean()
movies_matrix["rating_average"] = movies_matrix["rating_average"].
 ↪fillna(global_mean)
```

[74]:
```
print(movies_matrix.head())
```

```
   movie_id        movie_title  release_year                    genre_list  \
0         1   Toy Story (1995)        1995.0  [Animation, Children, Comedy]
1         2   GoldenEye (1995)        1995.0  [Action, Adventure, Thriller]
2         3  Four Rooms (1995)        1995.0                     [Thriller]
3         4  Get Shorty (1995)        1995.0       [Action, Comedy, Drama]
4         5     Copycat (1995)        1995.0      [Crime, Drama, Thriller]

  runtime_category  rating_average
0           medium        3.878319
1           medium        3.206107
2           medium        3.033333
3           medium        3.550239
4           medium        3.302326
```

[75]:
```
print("movies_matrix:", len(movies_matrix))
```

```
movies_matrix: 1664
```

[76]:
```
print("global_mean:", global_mean)
```

```
global_mean: 3.52986
```

[77]:
```
print("movies_matrix shape:", movies_matrix.shape)
```

```
movies_matrix shape: (1664, 6)
```

[78]:
```
print(movies_matrix["release_year"].describe())
```

```
count    1663.000000
mean     1988.986170
std        14.354978
min      1922.000000
25%      1992.000000
50%      1995.000000
75%      1996.000000
max      1998.000000
Name: release_year, dtype: float64
```

[79]:
```
print(movies_matrix["rating_average"].describe())
```

```
count    1664.000000
mean        3.077480
std         0.780919
min         1.000000
25%         2.665094
50%         3.162132
75%         3.652592
max         5.000000
Name: rating_average, dtype: float64
```

[80]:
```
print(movies_matrix["runtime_category"].describe())
print("\n")
print(movies_matrix["runtime_category"].value_counts())
```

```
count       1664
unique         4
top       medium
freq        1065
Name: runtime_category, dtype: object


runtime_category
medium     1065
long        359
short       213
unknown      27
Name: count, dtype: int64
```

2.0.2 3, 4: Implement content-based scoring, Generate recommendations with explanations:

[81]:
```
# Create 3 different user preference profiles
# Note:          Year range: 1922-1998
#                Rating Threshold range: 1-5
#                Genres:
#                    Action | Adventure | Animation |
#                    Children's | Comedy | Crime | Documentary | Drama | Fantasy|
#                    Film-Noir | Horror | Musical | Mystery | Romance | Sci-Fi |
#                    Thriller | War | Western

user_profiles = [
    {
        "name": "Jalin",
        "preferred_genres": ["Horror","Thriller","Sci-Fi"],
        "year_min": 1970,
        "year_max": 2010,
```

```
        "rating_threshold": 5,
        "runtime_pref": "Long"
    },
    {
        "name": "Jayla",
        "preferred_genres": ["Drama","Romance"],
        "year_min": 1980,
        "year_max": 1999,
        "rating_threshold": 4.0,
        "runtime_pref": "Medium"
    },
    {
        "name": "Mom",
        "preferred_genres": ["Comedy","Children"],
        "year_min": 2000,
        "year_max": 2010,
        "rating_threshold": 3.0,
        "runtime_pref": "Any"
    }
]
```

[82]:
```
# Generate top 3 recommendations with explanations for each profile (content␣
 ↪based scoring)
for user_profile in user_profiles:
    preferred_genres = user_profile["preferred_genres"]
    year_min = user_profile["year_min"]
    year_max = user_profile["year_max"]
    rating_threshold = user_profile["rating_threshold"]
    runtime_pref = user_profile["runtime_pref"]

    recommendations = []

    for i, movies in movies_matrix.iterrows():
        # Genre matching (50% weight): Calculate overlap using set intersection:
 ↪ score = len(movie_genres  preferred_genres) / len(preferred_genres) * 40
        genres_matched = list(set(movies["genre_list"]) & set(preferred_genres))
        genres_score = (len(genres_matched) / len(preferred_genres)) * 40.0

        year_score = 30.0 # Year preference (30% weight)
        rating_score = 20.0 # Rating filter (20% weight)

        # Total score = sum of all components (max 100)
        total_score = genres_score + year_score + rating_score

        recommendations.append({
            "genre_score": genres_score,
            "year_score": year_score,
```

```
            "rating_score": rating_score,
            "total_score": total_score,
            "genres_matched": genres_matched,
            "runtime_cat": runtime_pref,
            "rating_average": movies["rating_average"],
            "movie_id": movies["movie_id"],
            "movie_title": movies["movie_title"],
            "release_year": movies["release_year"]
        })

        recommendation_matrix = pd.DataFrame(recommendations).
↪sort_values("total_score", ascending=False).head(3)

    # Return top 3 movies with explanations of why each was selected
    print(f"Top 3 recommendations for {user_profile["name"]}")
    if(recommendation_matrix.empty):
        print("No recommendations for", user_profile["name"])
    for recommendation in recommendation_matrix.iterrows():
        print(recommendation)
```

```
Top 3 recommendations for Jalin
(568, genre_score                              40.0
year_score                                     30.0
rating_score                                   20.0
total_score                                    90.0
genres_matched          [Horror, Thriller, Sci-Fi]
runtime_cat                                    Long
rating_average                             2.939394
movie_id                                        573
movie_title                  Body Snatchers (1993)
release_year                                 1993.0
Name: 568, dtype: object)
(660, genre_score                              40.0
year_score                                     30.0
rating_score                                   20.0
total_score                                    90.0
genres_matched          [Horror, Thriller, Sci-Fi]
runtime_cat                                    Long
rating_average                                 2.83
movie_id                                        665
movie_title                        Alien 3 (1992)
release_year                                 1992.0
Name: 660, dtype: object)
(182, genre_score                              40.0
year_score                                     30.0
rating_score                                   20.0
total_score                                    90.0
genres_matched          [Horror, Thriller, Sci-Fi]
```

```
runtime_cat                              Long
rating_average                       4.034364
movie_id                                  183
movie_title                     Alien (1979)
release_year                          1979.0
Name: 182, dtype: object)
Top 3 recommendations for Jayla
(1644, genre_score                        40.0
year_score                            30.0
rating_score                          20.0
total_score                           90.0
genres_matched          [Romance, Drama]
runtime_cat                         Medium
rating_average                         2.5
movie_id                              1662
movie_title          Rough Magic (1995)
release_year                        1995.0
Name: 1644, dtype: object)
(276, genre_score                         40.0
year_score                            30.0
rating_score                          20.0
total_score                           90.0
genres_matched          [Romance, Drama]
runtime_cat                         Medium
rating_average                    3.233333
movie_id                               278
movie_title          Bed of Roses (1996)
release_year                        1996.0
Name: 276, dtype: object)
(1418, genre_score                          40.0
year_score                            30.0
rating_score                          20.0
total_score                           90.0
genres_matched          [Romance, Drama]
runtime_cat                         Medium
rating_average                        2.75
movie_id                              1429
movie_title          Sliding Doors (1998)
release_year                        1998.0
Name: 1418, dtype: object)
Top 3 recommendations for Mom
(138, genre_score                         40.0
year_score                            30.0
rating_score                          20.0
total_score                           90.0
genres_matched        [Comedy, Children]
runtime_cat                            Any
rating_average                        2.78
```

```
movie_id                           139
movie_title       Love Bug, The (1969)
release_year                    1969.0
Name: 138, dtype: object)
(1325, genre_score                      40.0
year_score                      30.0
rating_score                    20.0
total_score                     90.0
genres_matched    [Comedy, Children]
runtime_cat                      Any
rating_average                   1.8
movie_id                        1336
movie_title            Kazaam (1996)
release_year                  1996.0
Name: 1325, dtype: object)
(242, genre_score                        40.0
year_score                        30.0
rating_score                      20.0
total_score                       90.0
genres_matched      [Comedy, Children]
runtime_cat                        Any
rating_average                2.439394
movie_id                           243
movie_title       Jungle2Jungle (1997)
release_year                    1997.0
Name: 242, dtype: object)
```

^ All code up to this point written by Jalin A. Brown ^